

Informe de Laboratorio 10

Tema: Replicación de datos en Sistemas Distribuidos

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	<i>Replicación de datos en Sistemas Distribuidos</i>				
NÚMERO DE PRÁCTICA:	10	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN	19/06/2026	HORA DE PRESENTACIÓN	23:59		
INTEGRANTES: • CHAMBILLA PERCA RICARDO • JUAN DIEGO GUTIERREZ CCAMA • RYAN FABIAN VALDIVIA SEGOVIA				NOTA:	
DOCENTE(s): <i>Mg. Maribel Molina Barriga</i>					

1. Documento de Arquitectura de Software (SAD) - FedEx Perú

Este documento describe el diseño de la arquitectura distribuida con replicación para la empresa FedEx Perú, orientada a mitigar las interrupciones operativas, inconsistencias en inventarios y desactualización en el seguimiento de mercancía perecedera en las sedes de Lima, Bogotá, Santiago y Ciudad de México.

1.1. Vista de Casos de Uso y Requerimientos del Sistema

El sistema propuesto aborda los siguientes requerimientos fundamentales:

- **Alta Disponibilidad:** Garantizar que las sedes regionales mantengan la operatividad local incluso ante la desconexión o falla de los servidores centrales.
- **Consistencia Eventual y Fuerte según el tipo de datos:** El seguimiento de temperatura y ubicación puede tolerar consistencia eventual, mientras que el control de inventarios demanda niveles más rigurosos de consistencia para evitar ventas duplicadas o desabastecimiento.
- **Reducción de Latencia:** Asegurar que las consultas locales en cada sede sean atendidas directamente por una réplica local.

1.2. Vista Lógica

La lógica de datos se divide según el dominio del dominio de negocio. La persistencia de datos utiliza un modelo híbrido:

- **Bases de datos Relacionales (PostgreSQL/MySQL):** Utilizadas para el historial de pedidos e inventarios consolidados donde el cumplimiento de propiedades ACID es imperativo.
- **Bases de datos NoSQL (Cassandra/MongoDB):** Ideales para el registro continuo de telemetría (temperaturas) y geolocalización de vehículos debido a su capacidad de escritura de alto rendimiento y tolerancia a fallos mediante replicación integrada.

1.3. Vista de Despliegue

Cada centro de distribución regional (Lima, Bogotá, Santiago, Ciudad de México) cuenta con infraestructura local y una pasarela de sincronización WAN. Un balanceador de carga global (GSLB) distribuye las solicitudes de los usuarios externos al nodo geográficamente más cercano.

2. Desarrollo de las Actividades Propuestas

2.1. Actividad 1: Identificación de Necesidades

A continuación, se tabulan los datos críticos, riesgos operativos actuales y los beneficios esperados al implementar una estrategia de replicación en FedEx Perú:

Categoría de Datos	Datos Específicos Susceptibles	Riesgos Actuales	Beneficios Esperados
Inventarios	Stock físico, ubicaciones en almacén, reservas.	Sobreventa, inconsistencia entre sedes, desabastecimiento de perecederos.	Consistencia global, prevención de quiebres de stock.
Seguimiento de Envíos	Estado del paquete, última sede registrada.	Clientes reciben estados contradictorios o desactualizados.	Trazabilidad unificada en tiempo real, consultas locales rápidas.
Telemetría de Almacenamiento	Temperaturas, alertas de refrigeración.	Pérdida de alertas de cadena de frío si se cae el nodo central.	Tolerancia a fallos local, registro ininterrumpido.
Ubicación de Vehículos	Coordenadas GPS en tránsito, estimación de llegada.	Retrasos en actualizaciones operativas de despacho.	Visualización fluida sin sobrecargar el servidor central.
Historial de Pedidos	Transacciones, facturación, detalles del cliente.	Pérdida de transacciones durante cortes de energía o caídas.	Resiliencia de transacciones, auditoría confiable.

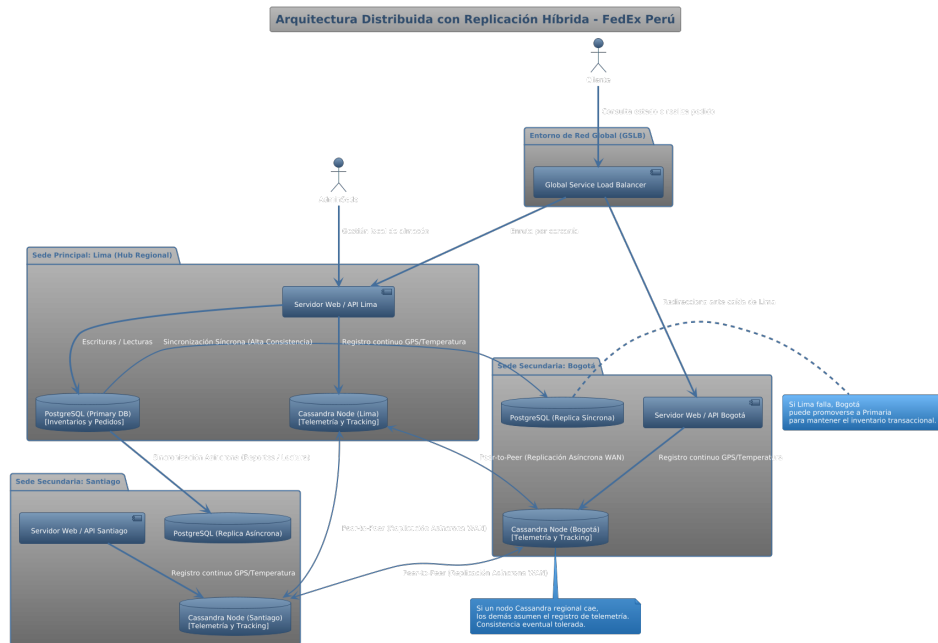
Tabla 1: Matriz de identificación de necesidades de replicación de FedEx Perú.

2.2. Actividad 2: Diseño Arquitectónico

La arquitectura propuesta se fundamenta en un esquema de topología híbrida que combina la replicación **Primary-Replica** para datos transaccionales, y replicación **Peer-to-Peer (NoSQL)** para el volumen masivo de lecturas/escrituras de telemetría y seguimiento.

2.2.1. Diagrama de Arquitectura

La Figura ?? describe de manera conceptual la interacción de los clientes mediante un balanceador de carga global, distribuyendo las solicitudes hacia las cuatro sedes de la organización.



2.2.2. Justificación de la Arquitectura

- **Escalabilidad Horizontal:** La persistencia no relacional (ej. Cassandra/MongoDB) permite agregar nodos adicionales en cualquiera de los países sin alterar el esquema de datos global.
- **Tolerancia a Desastres Regionales:** Al distribuir las copias de los datos de inventario y pedidos en múltiples ubicaciones físicas, el colapso total de una infraestructura regional no interrumpe el procesamiento global de pedidos.
- **Mitigación del Retraso de Red (Rendimiento local):** Al redirigir el tráfico de consulta de los clientes a réplicas de lectura locales en cada sede, la latencia disminuye sustancialmente respecto a una consulta transoceánica o transcontinental.

2.3. Actividad 3: Selección del Tipo de Replicación

La selección de las estrategias de replicación responde a los requerimientos específicos de cada tipo de flujo de datos, ponderando la consistencia, disponibilidad y latencia (Teorema de CAP).

Tipo de Datos	Estrategia de Replicación	Mecanismo de Sincronización	Justificación (Consistencia / Disponibilidad / Rendimiento)
Inventarios	Primary-Replica con Conmutación manual/automática	Síncrona	Consistencia Fuerte: Previene la venta de un mismo producto perecedero en tránsito a dos sedes distintas. Requiere validación inmediata.
Seguimiento de Envíos	Peer-to-Peer (Multi-Master distribuido)	Asíncrona	Alta Disponibilidad: Es preferible registrar la ubicación de un envío de inmediato de forma local, asumiendo un pequeño desfase de minutos para su reflejo en otras sedes.
Historial de Pedidos	Primary-Replica con nodo primario geográfico	Síncrona / Semisíncrona	Integridad de Datos: Asegura que las transacciones y cobros queden firmemente asentados en el sistema antes de emitir una confirmación al usuario.
Reportes Ejecutivos	Replica secundaria dedicada exclusivamente a lectura	Asíncrona diferida	Optimización del Rendimiento: Permite a los analistas de negocio procesar grandes consultas de agregación analítica (OLAP) sin degradar la base de datos operativa (OLTP).

Tabla 2: Tabla comparativa de estrategias de replicación seleccionadas para FedEx Perú.

2.4. Actividad 4: Simulación de un Fallo

Escenario: El centro de datos principal de Lima queda inoperativo durante 20 minutos debido a una falla masiva de conectividad local.

1. ¿Qué nodo debería asumir el servicio?

Para los datos transaccionales de inventarios y pedidos bajo control de PostgreSQL/MySQL, un nodo de respaldo previamente designado (por ejemplo, Bogotá, debido a su cercanía geográfica y baja latencia de red hacia Lima) debe ser promovido automáticamente a Primary a través de un orquestador de failover. Para los servicios NoSQL peer-to-peer (ej. Cassandra/MongoDB), el balanceador de carga global (GSLB) detecta la indisponibilidad de Lima y redirige de inmediato el tráfico de lectura y escritura a los nodos restantes en Bogotá o Santiago, los cuales continúan procesando sin necesidad de reconfiguración manual.

2. ¿Cómo se garantizaría la continuidad operativa?

La continuidad local para el personal en Lima se mantendrá de forma degradada temporalmente:

- Las aplicaciones de registro de telemetría y geolocalización local pueden acumular los eventos de forma asíncrona en una caché de cola de mensajes en memoria local (ej. Redis en un servidor secundario local de la oficina de Lima si tiene energía de respaldo) o desviar directamente las peticiones a la nube.
- El balanceador redirecciona el tráfico web externo a la sede operativa más cercana.

3. ¿Qué información podría perderse si la replicación fuera asíncrona?

Se podrían perder aquellos datos que estaban pendientes de sincronización en tránsito (*Replication Lag*) al momento de ocurrir el corte físico. En FedEx, esto se traduciría en:

- Lecturas de temperatura de las cámaras frigoríficas de Lima registradas en los últimos segundos antes del fallo.

- Actualizaciones del estado de un envío escaneado justamente en la ventana de desconexión.

Dichos datos quedarán huérfanos localmente y se resincronizarán únicamente cuando el nodo Lima vuelva a estar en línea, pudiendo causar inconsistencias temporales en el sistema global.

4. ¿Cómo afectaría una replicación síncrona?

Si todo el sistema dependiera de una replicación puramente síncrona:

- **Impacto negativo de latencia:** Durante la operación normal, cualquier actualización de temperatura en Bogotá tendría que esperar la confirmación de escritura física de Lima antes de completarse, lo que ralentizaría críticamente todos los procesos a nivel continental.
- **Bloqueo de escrituras:** Al caerse Lima, todas las transacciones de escritura en Bogotá, Santiago y México quedarían congeladas o fallarían inmediatamente al no recibir la confirmación (*ACK*) de la sede de Lima afectada. Esto provocaría una interrupción operativa total y no solo localizada.

2.5. Actividad 5: Evaluación Crítica

A fin de incrementar sustancialmente la resiliencia en la infraestructura distribuida de la corporación, se sugieren las siguientes mejoras tecnológicas:

1. Implementación de un Orquestador de Failover Activo-Activo con Base de Datos Distribuida Multiregión:

En lugar de esquemas tradicionales que dependen de scripts de failover propensos a fallar, el uso de motores distribuidos modernos con algoritmos de consenso como Raft o Paxos (ej. CockroachDB o clústeres nativos de Postgres coordinados mediante **Patroni** y **Consul**) reduce el tiempo de recuperación (RTO) a pocos segundos, asegurando que ninguna transacción confirmada se pierda (RPO = 0).

2. Uso de Cachés de Mensajería Tolerantes a Particiones en el Edge (Redis + Kafka):

Colocar brokers locales de **Apache Kafka** en cada una de las sedes permite mitigar la pérdida de conectividad WAN. En caso de desconexión del enlace principal, los sistemas de telemetría locales continúan enviando las coordenadas y temperaturas al broker Kafka local. Cuando el enlace de red se restablece, Kafka transmite de forma ordenada y secuencial los eventos retenidos a los clústeres centrales de base de datos.

3. Políticas de Enrutamiento Geográfico Inteligente mediante Anycast DNS y Service Mesh:

La adopción de tecnologías de malla de servicios (*Service Mesh* como Istio) combinadas con balanceadores DNS que utilicen direccionamiento Anycast asegura la redirección automatizada de solicitudes ante fallos del centro de datos. Si la API de la sede de Lima no responde al sondeo de salud (*Health Check*), el tráfico de red de los clientes locales se transfiere de inmediato a Bogotá de manera transparente para el usuario final.

3. Cuestionario

1. En una empresa multinacional con millones de transacciones diarias, ¿cómo equilibraría la necesidad de consistencia fuerte con los requisitos de disponibilidad y baja latencia? Sustente su respuesta con ejemplos.

El balance ideal de este escenario se gestiona aplicando el **Teorema PACELC** (una extensión del Teorema de CAP). No todos los datos del negocio demandan el mismo nivel de consistencia. El equilibrio se alcanza segregando los microservicios y aplicando arquitecturas como CQRS (Command Query Responsibility Segregation):

- **Ejemplo 1 (Consistencia Fuerte y Latencia Aceptable):** El saldo bancario o la reserva de stock real debe asegurar consistencia fuerte. Se utiliza replicación síncrona en clústeres locales y se acepta una latencia ligeramente superior (por ejemplo, transacciones de base de datos distribuidas con commit en dos fases en nodos de la misma región).
- **Ejemplo 2 (Consistencia Eventual y Baja Latencia):** El feed de notificaciones, la geolocalización del paquete o las valoraciones de servicios no requieren consistencia instantánea. Se implementa replicación asíncrona local de lectura inmediata. El usuario percibe una interfaz extremadamente veloz mientras que el sistema centralizado sincroniza las bases de datos en segundo plano.

2. ¿Es recomendable utilizar replicación síncrona para todos los sistemas empresariales? Analice los posibles impactos técnicos, económicos y operativos de esa decisión.

No es recomendable. Aplicar replicación síncrona de manera global e indiscriminada tiene consecuencias severas:

- **Impacto Técnico:** Degradación drástica del rendimiento. La velocidad de escritura se ve limitada por el nodo más lento del sistema y por la latencia de la red que conecta las réplicas (efecto cuello de botella). Adicionalmente, el sistema pierde tolerancia ante fallos de conectividad, bloqueando operaciones si se interrumpe la conexión WAN.
- **Impacto Económico:** Demanda una inversión extremadamente alta en infraestructura de red de banda ancha de ultra baja latencia (enlaces dedicados de fibra óptica) y servidores de alto rendimiento que soporten concurrencia sin bloquearse.
- **Impacto Operativo:** Caídas frecuentes de los servicios de cara al cliente ante la mínima inestabilidad del enlace internacional, afectando negativamente la reputación de la organización y el cumplimiento de los Acuerdos de Nivel de Servicio (SLA).

3. Si usted fuera el arquitecto de software de FedEx Perú, ¿qué estrategia de replicación implementaría para garantizar continuidad del negocio ante desastres regionales y por qué?

Implementaría una **Estrategia de Replicación Híbrida Geodistribuida Multi-Nube:**

- **Para Datos Críticos (Inventarios y Pedidos):** Replicación **Semisíncrona** entre la sede local y un clúster secundario en la nube en la región más cercana de AWS o Azure. El nodo principal escribe localmente de forma ultra-rápida y espera confirmación de solo un nodo cercano en la nube antes de validar la transacción. Esto protege los datos transaccionales contra desastres de infraestructura local sin introducir latencias prohibitivas hacia otros continentes.
- **Para Telemetría y Trazabilidad de Rutas:** Un esquema NoSQL distribuido (como Cassandra) con factor de replicación distribuido en las 4 sedes regionales. Cassandra tolera la caída completa de sedes geográficas enteras y permite que las sedes sanas sigan aceptando lecturas y escrituras, resincronizando automáticamente al nodo dañado tras su reincorporación.

4. Conclusiones

4.1. Conclusión de CHAMBILLA PERCA RICARDO

El diseño arquitectónico para sistemas de alta disponibilidad y volumen transaccional como el de FedEx requiere una rigurosa separación de responsabilidades y tipologías de datos. Descubrir que la replicación no es un enfoque binario (síncrono o asíncrono) para toda la base de datos, sino un diseño modular ajustado al valor estratégico de cada registro, es fundamental para construir sistemas de software distribuidos que sean tanto veloces como confiables.

4.2. Conclusión de JUAN DIEGO GUTIERREZ CCAMA

La aplicación pragmática de los teoremas de CAP y PACELC nos obliga, como ingenieros, a aceptar la realidad física de la latencia de red. El uso de bases de datos relacionales tradicionales debe limitarse a núcleos transaccionales estrictamente controlados, mientras que la telemetría de dispositivos de cadena de frío y geolocalización debe recaer sobre modelos NoSQL que favorezcan la alta disponibilidad local, reduciendo significativamente los puntos de falla global de la compañía.

4.3. Conclusión de RYAN FABIAN VALDIVIA SEGOVIA

La tolerancia a desastres no se logra únicamente implementando múltiples servidores redundantes, sino diseñando procesos automáticos de conmutación de tráfico y failover que reduzcan la intervención humana. El despliegue de mecanismos como colas de mensajería asíncronas en el Edge garantiza que el negocio siga operando de manera local temporalmente aislada, evitando que una desconexión WAN parcial paralice las actividades logísticas de distribución internacional.

5. Referencias

- [1] Tanenbaum, A. S. y Van Steen, M. (2008). *Sistemas distribuidos: principios y paradigmas*. México. Pearson Educación.
- [2] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2nd Edition.
- [3] PostgreSQL Documentation (2026). *Chapter 27. High Availability, Load Balancing, and Replication*. Recuperado de <https://www.postgresql.org/docs/current/high-availability.html>
- [4] Apache Cassandra Documentation (2026). *Data Replication in Cassandra*. Recuperado de <https://cassandra.apache.org/doc/latest/>
- [5] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

A. Diagrama de la Arquitectura Distribuida en PlantUML

A continuación se presenta el modelado de la arquitectura como un diagrama de arquitectura de software

